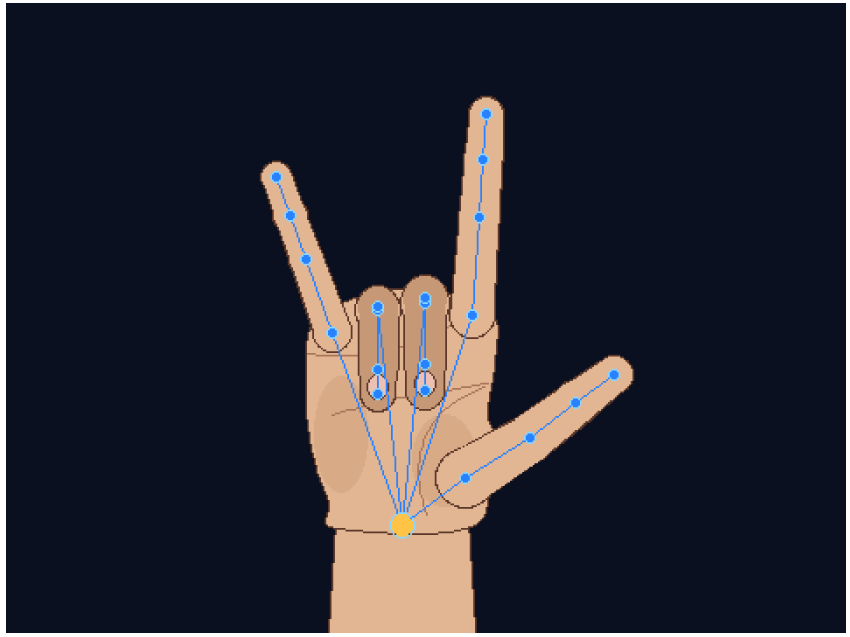
Sign Language Gesture Visualizer

Checkpoint 1 - Technical Foundation: graphical output evidence

Computer Graphics semester project | 6 October 2026

An interactive visualizer of American Sign Language handshapes on a rigged hand. For Checkpoint 1 the hand is built from lines, circles, ellipses, polygons and curves, all drawn by our own algorithms in **pure C++**: the program uses only the C++ standard library, draws every pixel itself and saves the images with its own .bmp writer. Every image in this document was produced that way.



Group members

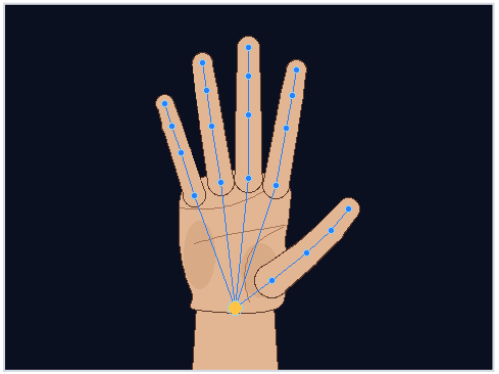
Abdalla Muhammad	169962	Hand rig, forward kinematics, drawing the hand, build setup
Elvis Wafuke	154210	Lines (DDA, Bresenham, attributes), Bezier curves, splines, algorithm pages
Ilhan Hamud	169004	Framebuffer and BMP output, polygon operations, circles, ellipses
Joy Wanjohi	159540	Scan-line fill, line and polygon clipping, interactive viewer
Benson Gitonga	156740	Sign library, Checkpoint 1 renderer, testing, documentation

Techniques implemented and where they appear in the hand

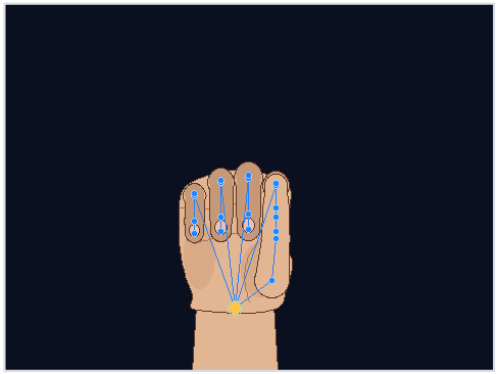
Lines	DDA, Bresenham; dashed, dotted and thick lines	Skeleton bones, outlines, motion trails
Circles	Midpoint circle (8-way symmetry), filled	Knuckles, fingertips, joints
Ellipses	Midpoint ellipse (two regions), filled	Palm pads, fingernails
Polygons	Scan-line fill (Edge Table + Active Edge Table); area, convexity, point-in-polygon, centroid	Forearm, palm, finger bones
Clipping	Cohen-Sutherland (lines), Sutherland-Hodgman (polygons), point clipping, window-to-viewport	Everything; the detail view
Curves	Bezier (de Casteljau, Bernstein form)	Palm crease lines
Splines	Catmull-Rom, uniform cubic B-spline	Palm outline, fingertip motion trails

1. The hand: 8 ASL handshapes

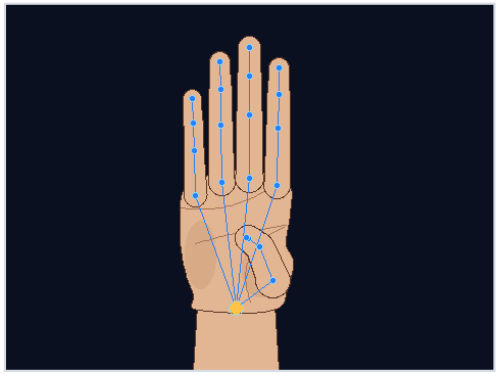
Each sign is a list of joint angles; forward kinematics places the joints and the primitives below draw the hand.



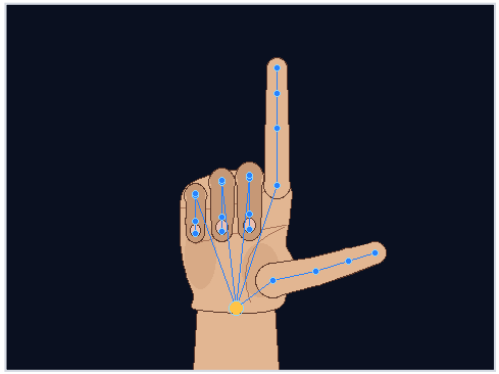
Sign "5"



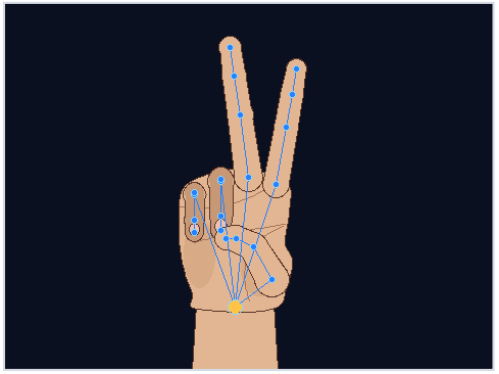
Sign "A"



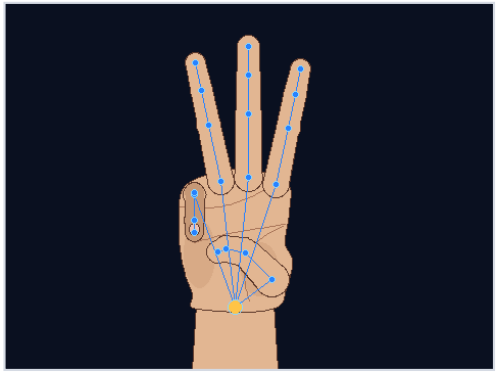
Sign "B"



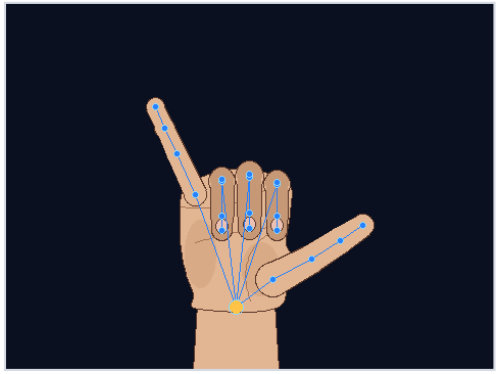
Sign "L"



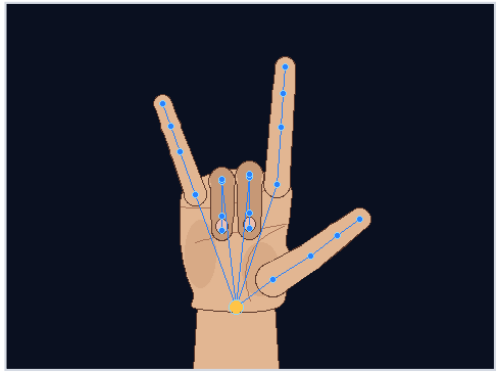
Sign "V"



Sign "W"



Sign "Y"

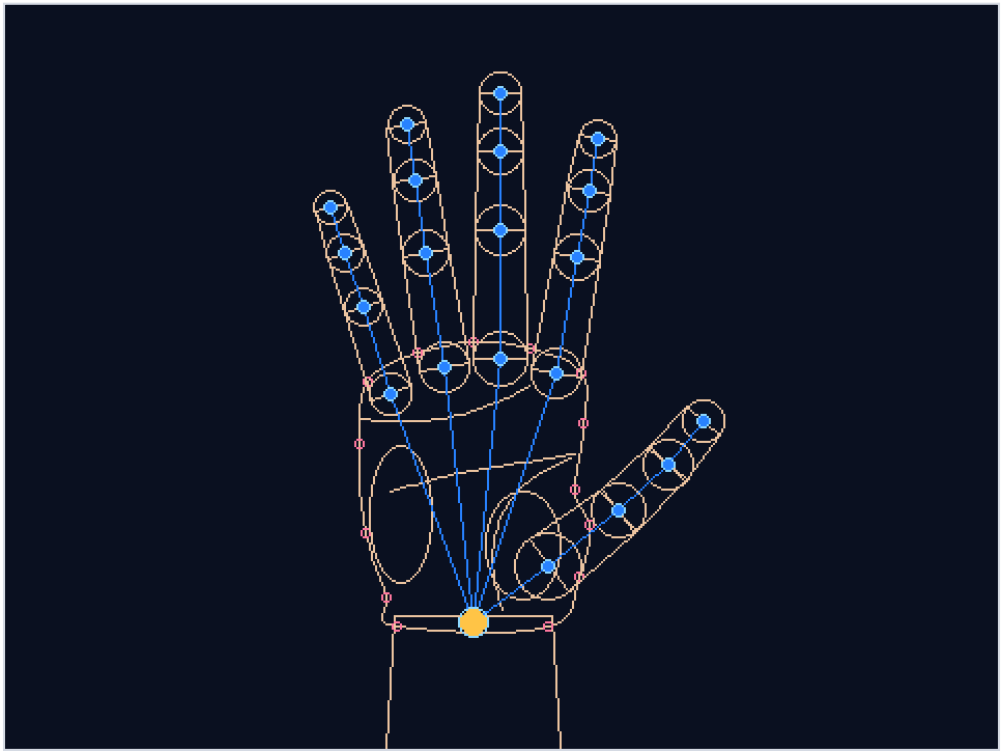


Sign "I love you"

Built from: scan-line filled polygons (forearm, finger bones), a closed Catmull-Rom spline (palm outline), midpoint circles (knuckles, fingertips, joints), midpoint ellipses (palm pads; nails on curled fingers), cubic Bezier curves (palm creases) and Bresenham lines (skeleton). Everything is clipped to the screen with Cohen-Sutherland and Sutherland-Hodgman before it is drawn.

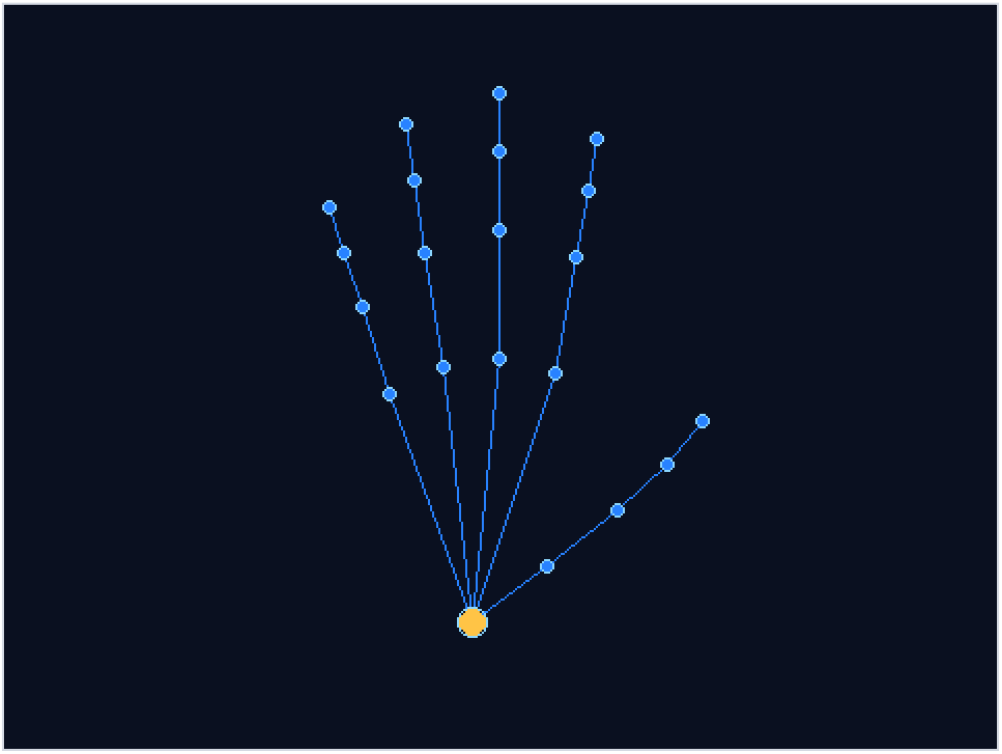
2. The hand: how it is built

The same open hand with the fill turned off, and with only the joint hierarchy shown.



Wireframe: every primitive outlined

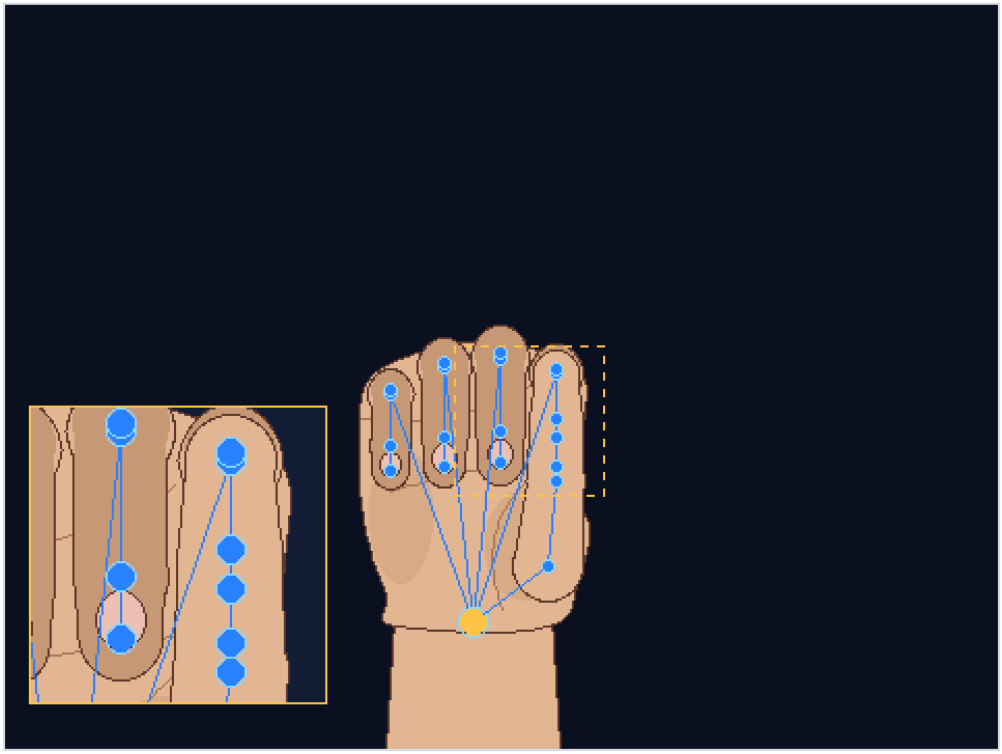
Finger bones are 4-sided polygons with midpoint circles at each end. The two ellipses are the palm pads. The palm outline is a closed Catmull-Rom spline; its 14 control points are the small pink circles. The three curved lines on the palm are cubic Bezier curves.



Skeleton: the joint hierarchy

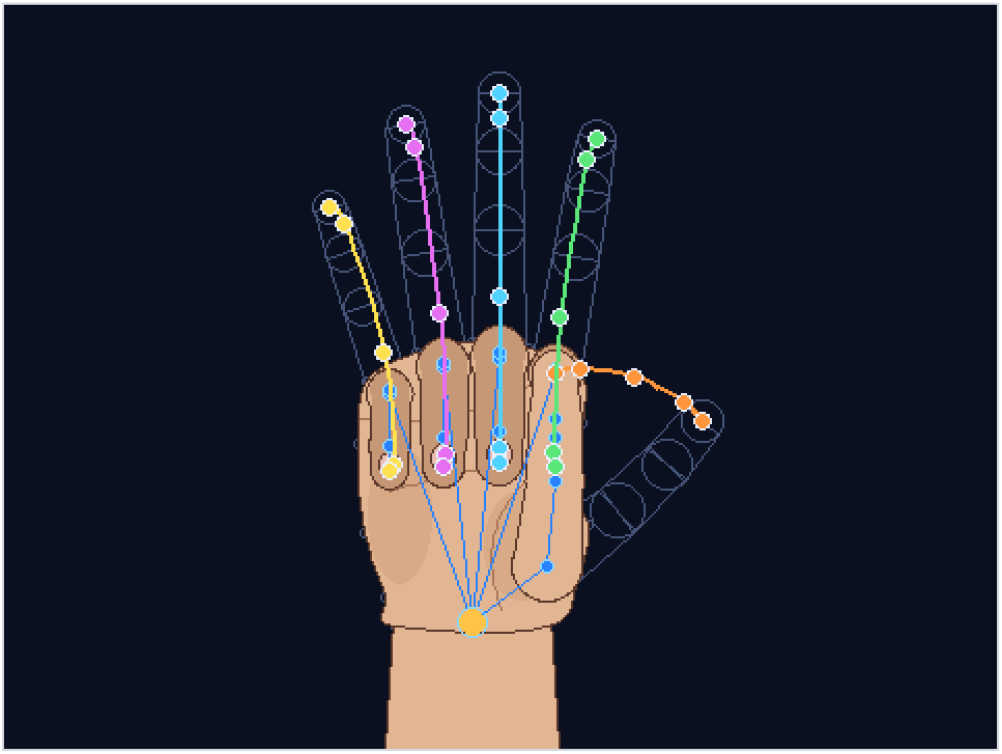
The wrist (yellow) is the root. Each finger has three joints and a fingertip, joined by Bresenham lines; joints are filled circles with a midpoint-circle ring. Rotating a joint moves every joint after it (forward kinematics).

3. The hand: clipping and splines at work



Detail view: clipping + window-to-viewport (sign "A")

The dashed rectangle is the window. The same hand is drawn again 2x larger into the viewport at the left, using x_v = V.xmin + (x_w - W.xmin) k. Lines are clipped to the viewport with Cohen-Sutherland, filled shapes with Sutherland-Hodgman, circles and ellipses by point clipping, so nothing spills onto the main picture. The magnified fingernails are filled midpoint ellipses.

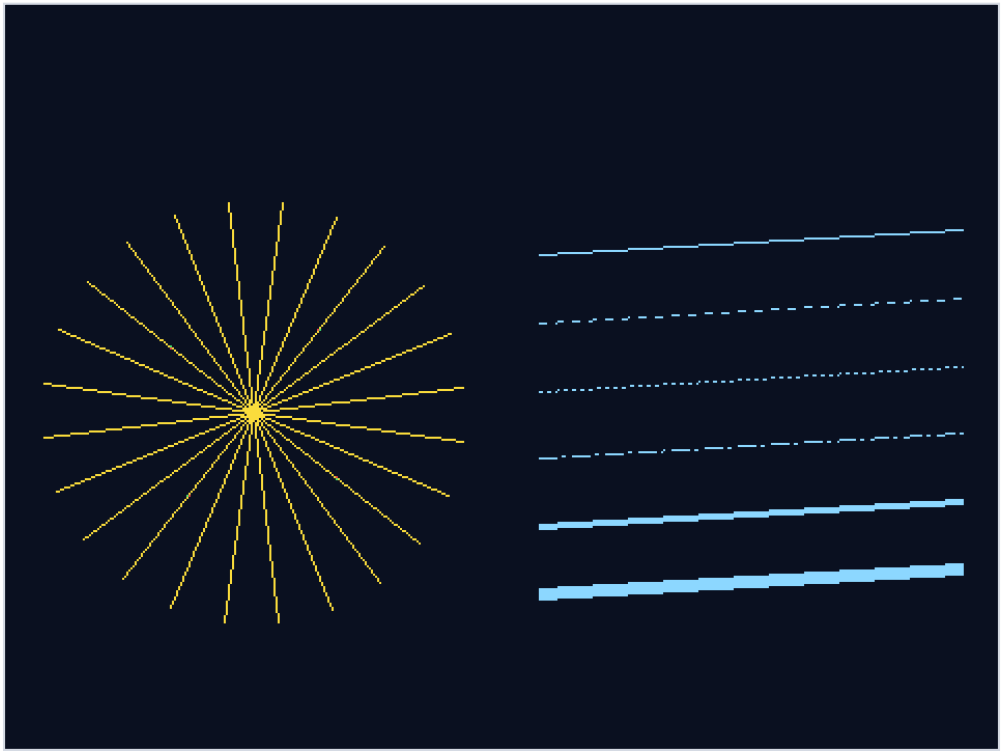


Motion trails: Catmull-Rom splines (sign 5 to sign A)

The dim outline is the start pose. Each fingertip was recorded at 5 keyframes of the movement (dots); an open Catmull-Rom spline through those keyframes, drawn with thick lines, shows its path. This prepares the keyframe animation of Checkpoint 2.

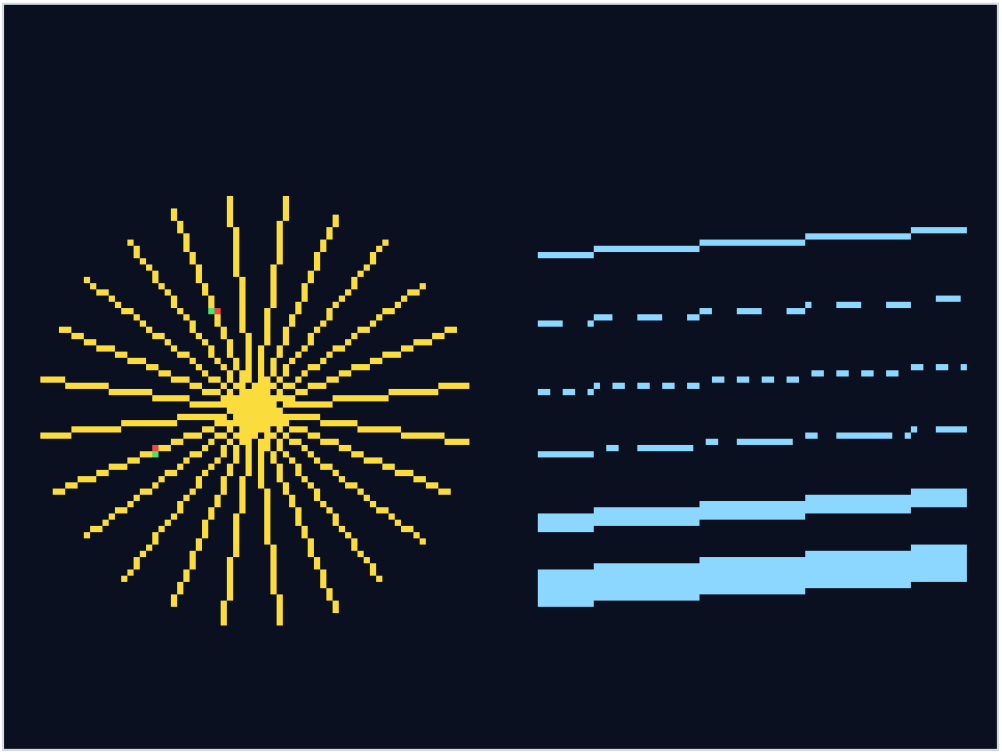
4. Lines and line operations

page_1_lines.bmp and its zoomed version



DDA vs Bresenham, and line attributes

Left: 24 lines in all 8 octants drawn by both algorithms; yellow = both chose the pixel, red = DDA only, green = Bresenham only. They differ on 8 of the pixels, at near-ties where the DDA's floating-point error tips its rounding. Right, top to bottom: solid, dashed (pattern 0x0F0F), dotted (0x3333), dash-dot (0x19FF), and thick lines of width 3 and 6 (pixel replication).

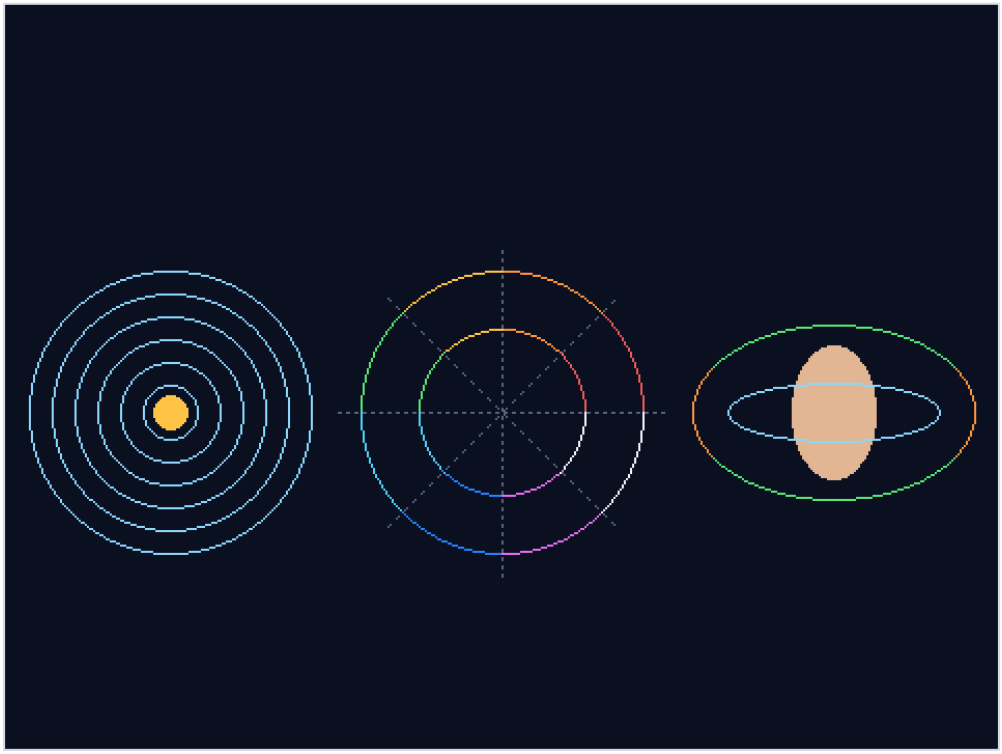


The same page at 6x pixel size

Each square is one pixel. The staircase of every line and the on/off pattern of the dashed lines are visible.

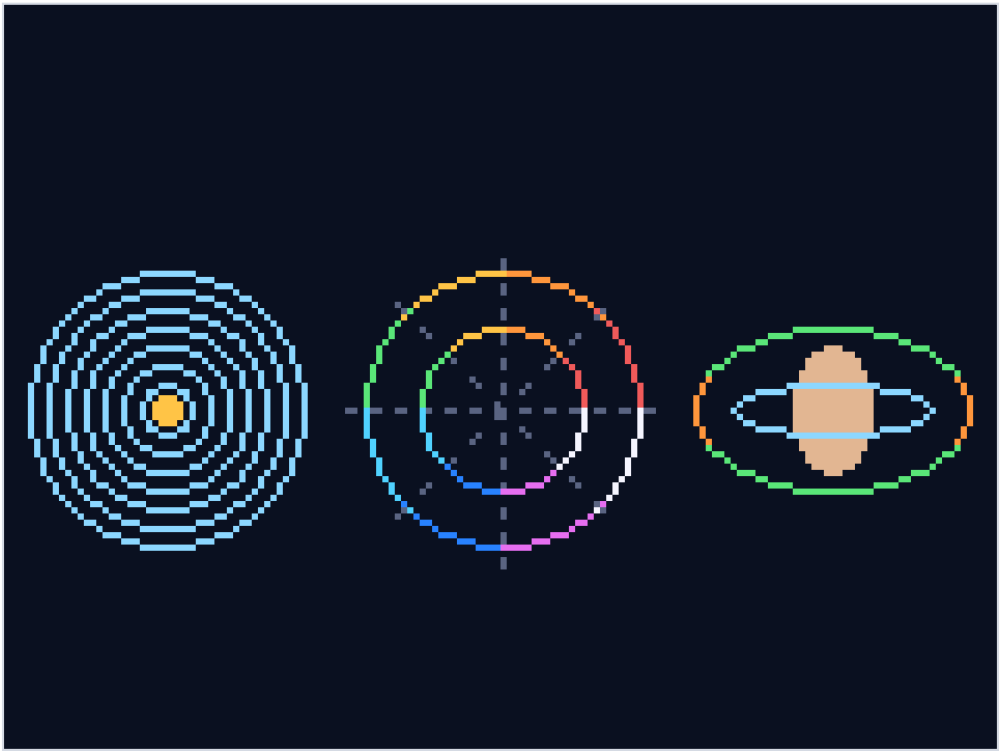
5. Circles and ellipses

page_2_circles_ellipses.bmp and its zoomed version



Midpoint circle, 8-way symmetry, midpoint ellipse

Left: concentric midpoint circles and a filled circle. Middle: only one octant is computed; each of the 8 mirrored copies has its own colour. Right: midpoint ellipse with region 1 (slope above -1) in green and region 2 in orange, a filled ellipse and a flat ellipse.

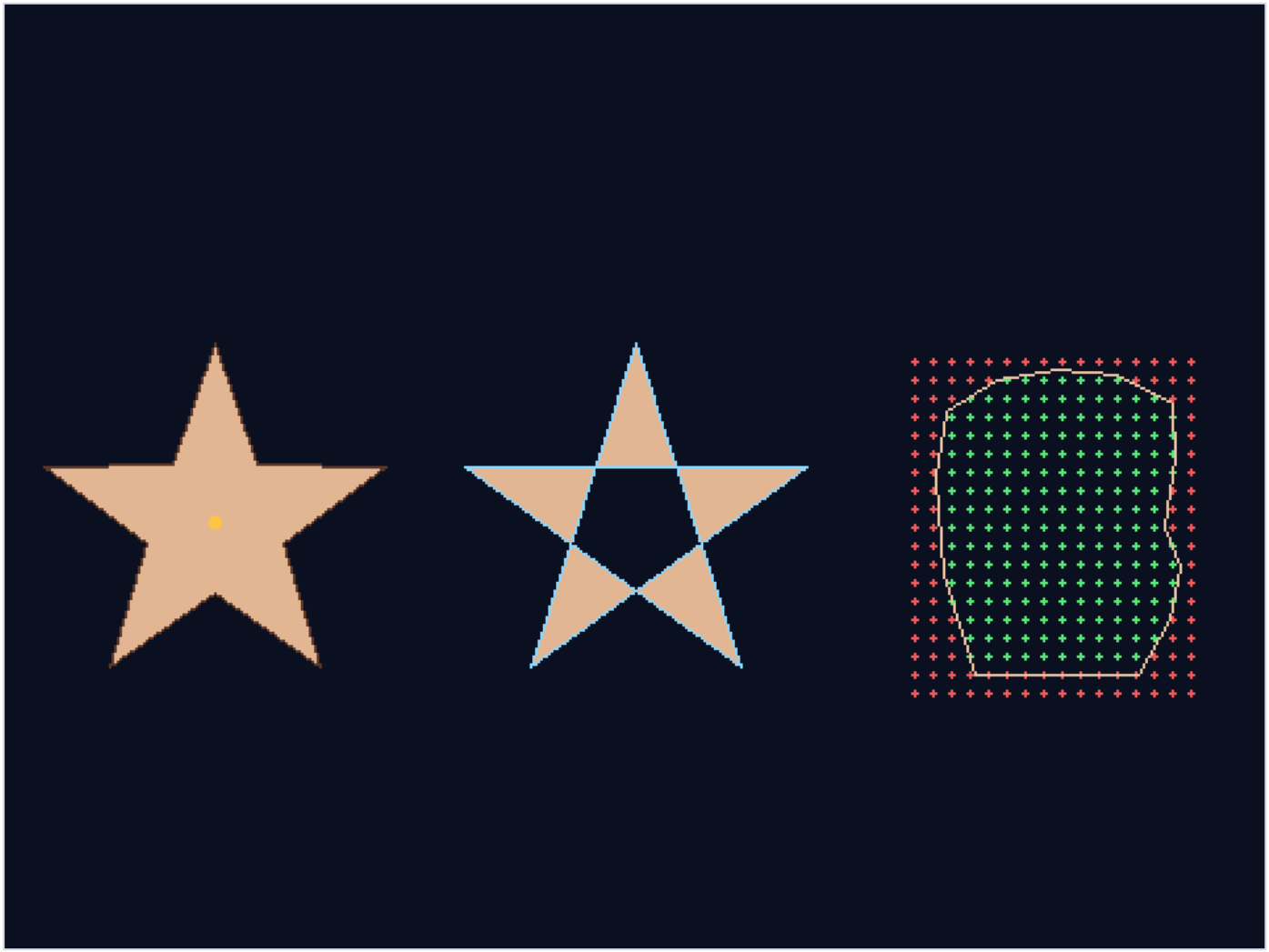


The same page at 6x pixel size

The ellipse shows where region 1 hands over to region 2: steps along x turn into steps along y.

6. Polygons: filling and polygon operations

page_3_polygons.bmp



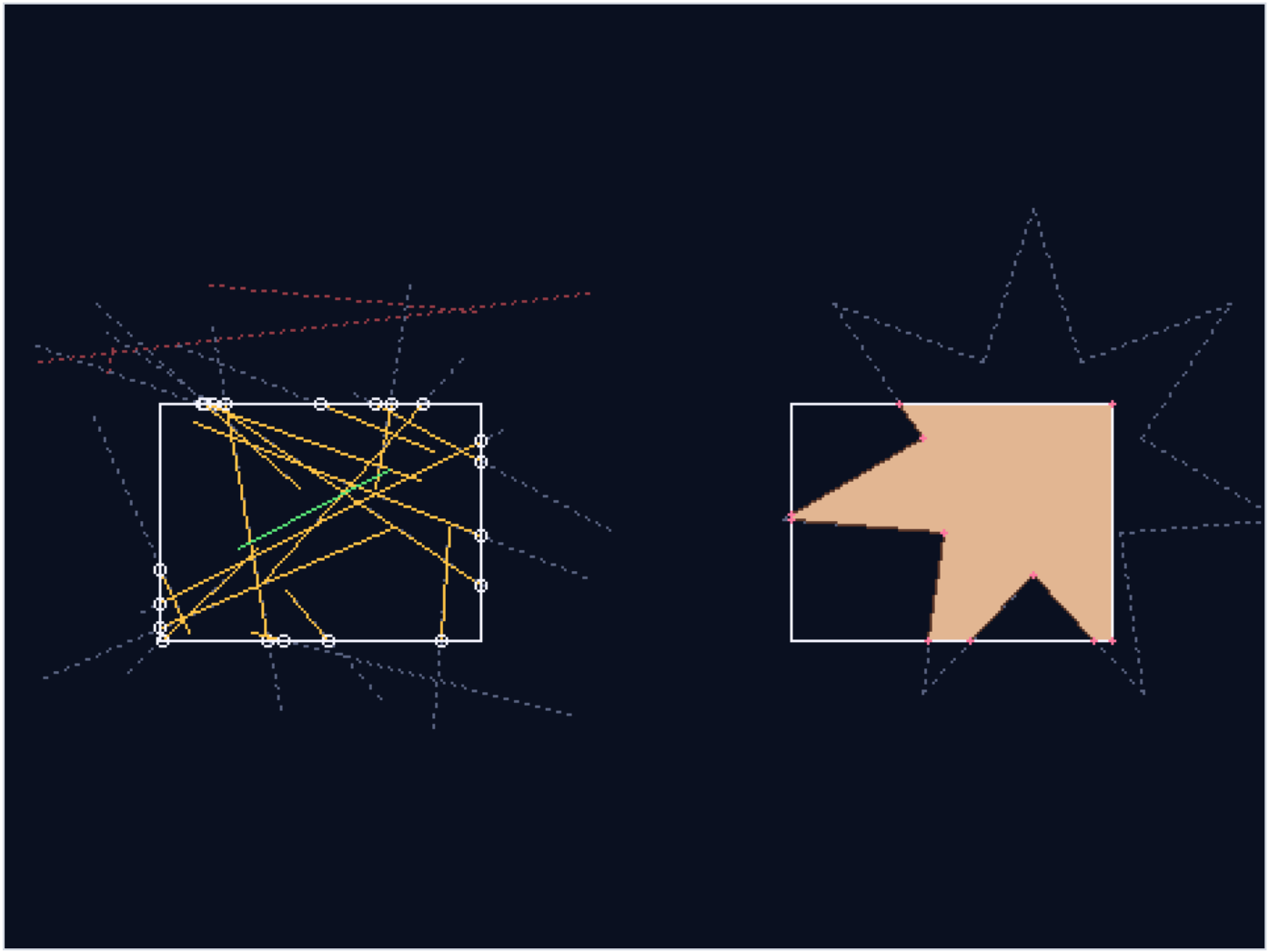
Left - scan-line fill of a concave star. An Edge Table and an Active Edge Table give the crossings on each row; the spans between pairs of crossings are filled. The dot is the centroid.

Middle - even-odd rule. A self-intersecting star (pentagram): the centre is crossed twice, so it is outside and stays empty.

Right - point-in-polygon on the palm shape from our hand model. Every dot was tested with the even-odd ray-casting test: green = inside, red = outside. The palm is concave at the web of the thumb, and its area is computed with the shoelace formula.

7. Clipping

page_4_clipping.bmp



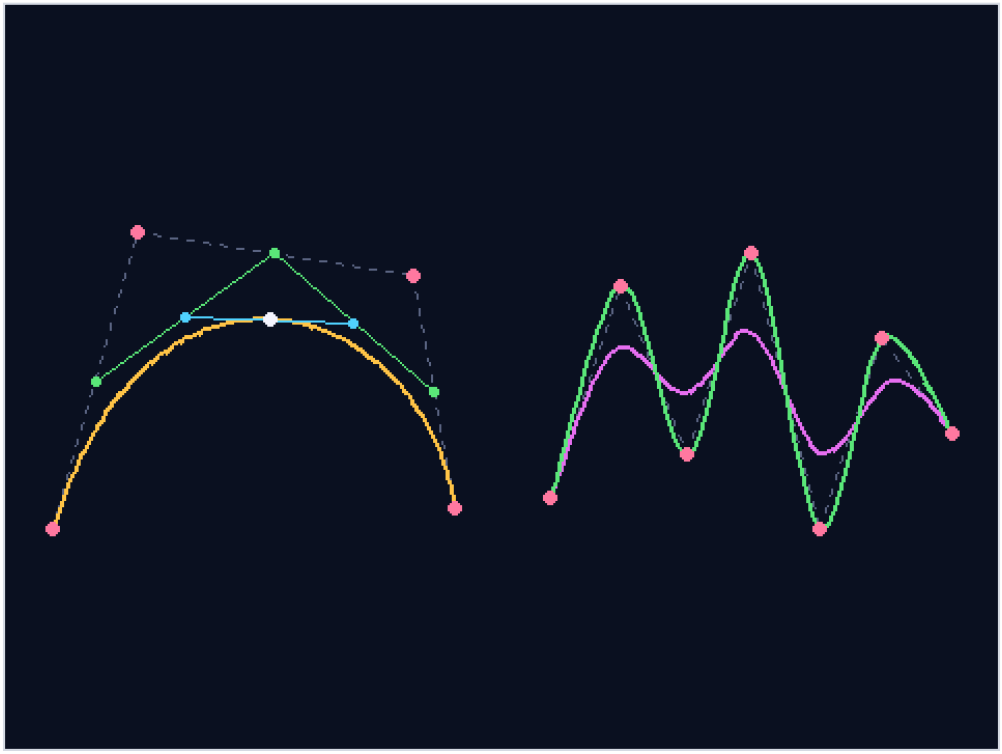
Left - Cohen-Sutherland line clipping. 20 lines against the white window. Dashed = the original line; solid green = accepted whole; solid yellow = clipped; red dashed = rejected (both end points beyond the same side, found from the 4-bit outcodes). White rings mark end points moved onto the window edge. Result: 1 whole, 16 clipped, 3 rejected.

Right - Sutherland-Hodgman polygon clipping. A 14-vertex concave star (dashed) is clipped against the left, right, bottom and top window edges in turn, giving an 11-vertex polygon (pink dots), which is then scan-line filled.

In the hand, every line and filled shape goes through these two algorithms before it is drawn (see the detail view in section 3).

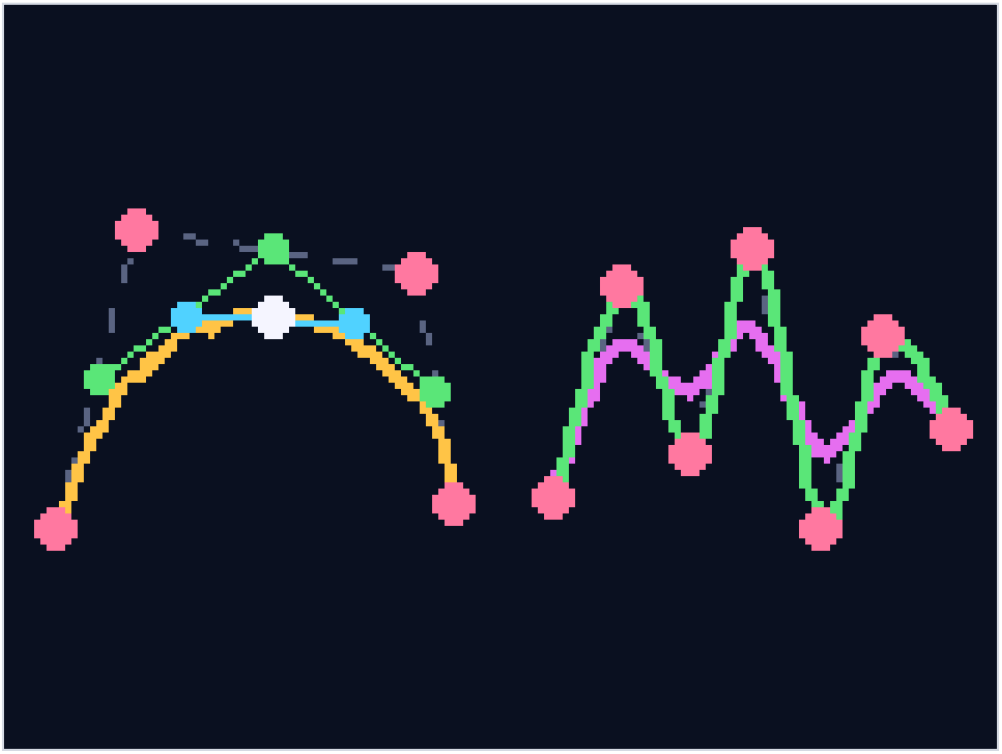
8. Curves: Bezier curves and splines

page_5_curves.bmp and its zoomed version



Cubic Bezier with de Casteljau construction; Catmull-Rom vs B-spline

Left: a cubic Bezier curve (yellow) with its control points (pink) and control polygon (dashed). The green and blue lines are the levels of de Casteljau's construction at t = 0.5; the white dot is P(0.5). Right: the same 7 points joined by a Catmull-Rom spline (green), which passes through every point, and a uniform cubic B-spline (pink), which is smoother and stays inside the control polygon.

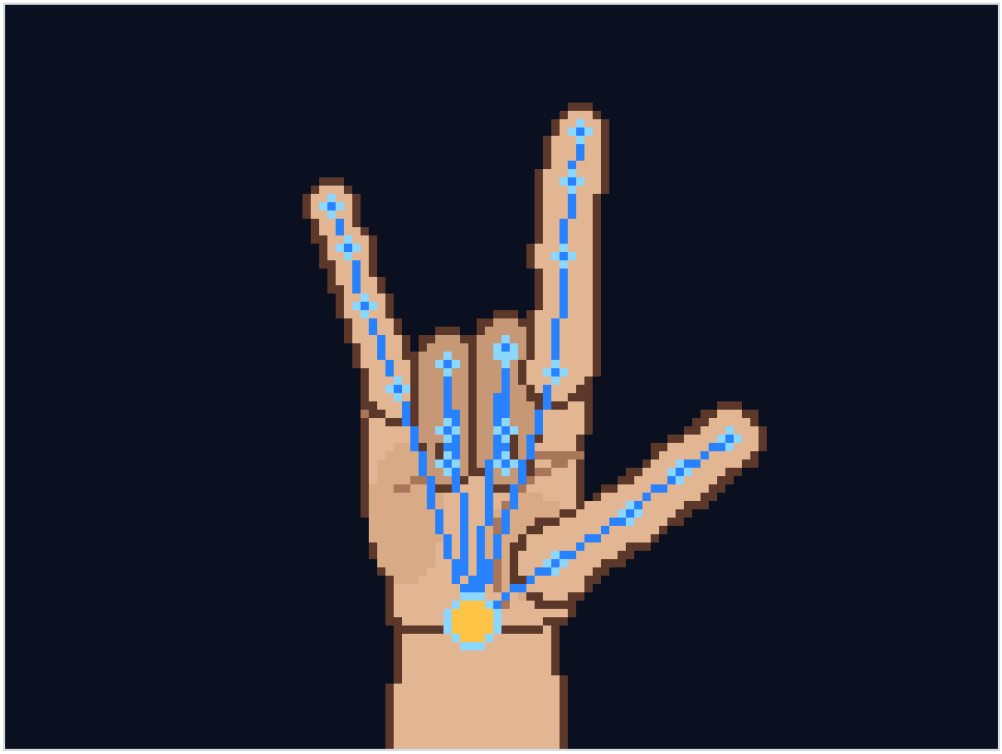


The same page at 6x pixel size

Curves are drawn by evaluating P(t) at many values of t and joining the points with lines.

9. Pixel level and testing

hand_pixels_8x.bmp



The hand at 120 x 90 pixels, each shown 8x larger

The steps produced by scan-line fill, the midpoint circles and the Bresenham lines are visible.

Automated tests (tests/render_test.cpp): **62 checks, all passing**, including
- the Bresenham, midpoint circle (r = 10) and midpoint ellipse (rx = 8, ry = 6) worked examples, pixel by pixel;
- lines in every direction, zero-length lines, dashed patterns and thick-line widths;
- exact fill counts, concave and self-intersecting polygons, shoelace area, convexity, point-in-polygon;
- Cohen-Sutherland accept / reject / clip cases and Sutherland-Hodgman inside / outside / overlap cases;
- 40 random lines and 40 triangles drawn through a clip window with no pixel landing outside it;
- de Casteljau = Bernstein form, the Bezier P(0.5) example, splines passing through (or near) their points;
- every sign renders, and the detail view never draws outside its viewport.

How these images were made: `build.bat`, then `bin\cp1_render.exe`, which writes the 21 .bmp files these images come from.